

Kalshi Crypto Trading Bot — Technical Whitepaper

Gabriel Kagan

February 2026

Contents

Part 1: Executive Summary	2
What It Does	2
Market Opportunity	2
Strategy in Plain English	3
Key Differentiators	3
Part 2: System Architecture	3
Data Pipeline	3
Component Overview	4
Data Sources	5
Part 3: Technical Deep-Dive	6
3.1 Volatility Engine	6
Realized Kernel (Barndorff-Nielsen 2008)	6
Mincer-Zarnowitz R ² -Weighted Blending	6
EGARCH(1,1) with Student-t Innovations (Shadow Mode)	7
HAR-RV Model (Shadow Mode)	7
Adaptive Jump Detection	7
DVOL Integration	7
Cross-Asset Beta	7
3.2 Probability Model	8
Step 1: Z-Score Computation	8
Step 2: NIG CDF (Per-Asset Fitted)	8
Step 3: Data-Driven Calibration	8
Step 4: Dynamic Probability Cap	9
Step 5: Market-Price Blending	9
Sanity Checks	9
3.3 Edge Detection	9
Fee Formula	9
Fee-Adjusted Edge	10
3.4 Order Flow Analysis	10
Cross-Exchange Consensus	10
Funding Rate Signals	10
Kalshi Order Flow (Shadow Mode)	10
Total Adjustment Cap	11

3.5 Execution Strategy	11
Three-Tier Post-Only Handler	11
Time-Based Escalation	11
Maker-to-Taker Conversion	11
Partial Fill Handling	12
UUID Persistence	12
3.6 Position Sizing	12
Quarter-Kelly Formula	12
Edge-Based Sizing Tiers	12
Drawdown Scaling	12
Part 4: Risk Management	13
Position Sizing Controls	13
Market Selection Controls	13
Model Sanity Controls	13
Execution Controls	13
Part 5: Performance & Observations	14
Overview	14
Filter Pipeline Breakdown	14
Hypothetical Performance	14
Win Rate by Price Bucket	15
Part 6: Infrastructure	15
Deployment	15
Data Persistence	15
SQLite (state.db)	15
JSONL Journals	15
Firebase Real-Time Dashboard	16
Shadow Mode Features	16

Part 1: Executive Summary

What It Does

This system is an automated trading bot for **Kalshi**, a CFTC-regulated prediction market exchange. It trades **15-minute cryptocurrency price threshold contracts** — binary options that pay \$1 if a crypto asset (BTC, ETH, SOL, or XRP) stays above a given price at the end of a 15-minute window, and \$0 otherwise.

The bot monitors real-time prices across multiple exchanges, estimates the probability of each outcome using microstructure-aware volatility models, and places trades when its model identifies a statistical edge over the market price.

Market Opportunity

Kalshi lists 15-minute crypto contracts around the clock. Each window produces fresh contracts for four assets at multiple strike prices, creating hundreds of tradeable markets per day. Because

these are short-duration, binary-outcome instruments, mispricing tends to be small but frequent — an ideal environment for systematic, model-driven trading.

Strategy in Plain English

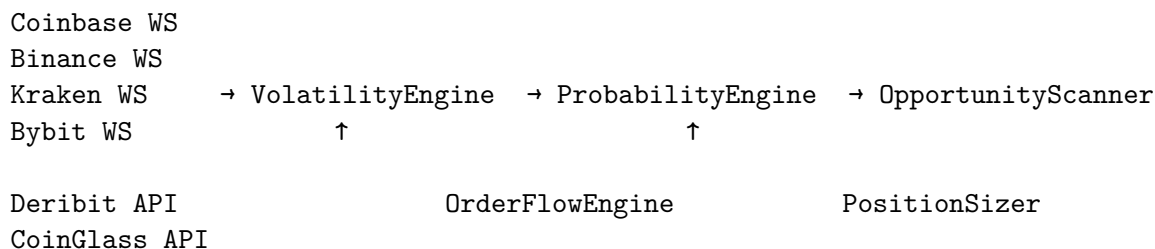
1. **Observe** — Continuously stream spot prices from Coinbase, Binance, Kraken, and Bybit. Fetch implied volatility from Deribit and funding rates from CoinGlass. Monitor Kalshi's own orderbook via WebSocket.
2. **Estimate** — For every active market, compute the probability that the asset stays above its threshold using a volatility-weighted, fat-tailed statistical model with per-asset Normal Inverse Gaussian (NIG) distributions.
3. **Filter** — Reject markets that are too uncertain, too expensive, or offer insufficient edge after fees.
4. **Size** — Use a conservative Kelly criterion (quarter-Kelly) to determine position size, with automatic scaling during drawdowns.
5. **Execute** — Place maker (limit) orders first to minimize fees, with three-tier post_only rejection handling and time-aware taker escalation.
6. **Settle** — Track outcomes via the Kalshi settlements API and log performance for continuous evaluation.

Key Differentiators

- **Multi-exchange intelligence:** Aggregates spot prices from 4 exchanges plus derivatives signals from Deribit and CoinGlass, detecting cross-exchange lead-lag patterns before they appear in Kalshi prices.
- **Microstructure-aware volatility:** Uses Realized Kernel estimation (Barndorff-Nielsen 2008) with data-adaptive bandwidth selection, Mincer-Zarnowitz R²-weighted blending, and EGARCH conditional volatility modeling.
- **Per-asset NIG distributions:** Normal Inverse Gaussian CDF replaces the generic Student-t, capturing both heavy tails and asymmetry specific to each cryptocurrency.
- **Adaptive execution:** Three-tier post_only rejection handler (normal → degraded → taker IOC), plus maker-first strategy with time-aware escalation, minimizes fees while ensuring fills.
- **Comprehensive risk controls:** Quarter-Kelly sizing, drawdown scaling, z-score sanity checks, data-driven calibration, and model-market discrepancy detection.

Part 2: System Architecture

Data Pipeline



OrderExecutor

Component	Role
HAREstimator	Heterogeneous Autoregressive RV model with extended variants (HAR-J, HAR-Semi, HAR-IV, HAR-VRP); fitted via ridge regression (shadow mode)
EGARCHEstimator	EGARCH(1,1) with Student-t innovations (df 3.2–3.8), MLE-fitted on 10,800 samples (3 hours), refitted hourly (shadow mode)
MZTracker	Mincer-Zarnowitz R^2 regression for dynamic EGARCH blend weight estimation with EMA smoothing
ProbabilityEngine	Win probability via NIG CDF (per-asset fitted) with data-driven calibration, dynamic caps, and market-price blending
CalibrationEngine	Learns calibration from settlement outcomes: Platt Scaling $\rightarrow$ Beta Calibration $\rightarrow$ Bayesian Linear Regression as data grows
PositionSizer	Edge-tiered position sizing with drawdown-based scaling
OpportunityScanner	Multi-stage filter pipeline evaluating all markets across active 15-minute windows
OrderExecutor	Three-tier post_only handler, maker-first limit orders with adaptive taker escalation, WebSocket fill detection, amend-first conversion
SettlementTracker	Incremental settlement polling (30-second intervals) using the Kalshi settlements API
TelegramNotifier	Optional Telegram alerts for trades, settlements, and errors
MainLoop	Continuous 1-second observation loop coordinating all components

Data Sources

Source	Data	Transport	Frequency
Coinbase	Spot prices (BTC, ETH, SOL, XRP)	WebSocket	Real-time (1s snapshots)
Binance	Spot prices (cross-exchange)	WebSocket	Real-time
Kraken	Spot prices (cross-exchange)	WebSocket	Real-time
Bybit	Spot prices (cross-exchange)	WebSocket	Real-time
Deribit	Implied volatility (DVOL) for BTC/ETH	REST API	60 seconds

Source	Data	Transport	Frequency
CoinGlass	Funding rates (BTC, ETH, SOL, XRP)	REST API	10 minutes
Kalshi	Markets, orderbooks, balance, fills, settlements	REST API + WebSocket	On-demand + real-time

Part 3: Technical Deep-Dive

3.1 Volatility Engine

The volatility engine produces a per-asset, per-5-second realized volatility estimate that feeds the probability model. It combines multiple techniques with data-adaptive weighting.

Realized Kernel (Barndorff-Nielsen 2008)

Standard sample variance of high-frequency returns is biased by market microstructure noise (bid-ask bounce, discrete tick sizes). The Realized Kernel estimator corrects this using a kernel-weighted autocovariance function with a Parzen flat-top kernel, producing noise-robust volatility estimates from 5-second log returns.

****Adaptive bandwidth (H^*)**:** Rather than using a fixed bandwidth $H=1$, the system estimates the optimal bandwidth from the data using the noise-to-signal ratio:

$$H^* = c \times \left(\frac{\hat{\omega}^2}{IV} \right)^{2/5} \times n^{3/5}$$

where $\hat{\omega}^2$ is the estimated microstructure noise variance and IV is the integrated variance. This produces tighter estimates during calm periods and wider smoothing during noisy periods.

Mincer-Zarnowitz R^2 -Weighted Blending

Rather than fixed weights (the earlier 50/30/20 scheme), the system dynamically weights estimators based on their forecasting quality. A Mincer-Zarnowitz regression compares each estimator's forecast against realized outcomes:

$$RV_{t+1} = \alpha + \beta \times \hat{\sigma}_t + \varepsilon_t$$

The R^2 from this regression measures forecast quality. Weights are smoothed using an EMA ($=0.97$) to prevent whipsawing:

$$w_t = \lambda \times w_{t-1} + (1 - \lambda) \times w_{raw}$$

Below an R^2 threshold of 0.10, the system reverts to equal-weight blending as a fallback.

EGARCH(1,1) with Student-t Innovations (Shadow Mode)

An EGARCH model captures volatility clustering and leverage effects:

$$\log(\sigma_t^2) = \omega + \alpha (|z_{t-1}| - E[|z|]) + \gamma z_{t-1} + \beta \log(\sigma_{t-1}^2)$$

Fitted with Student-t innovations (df typically 3.2–3.8 for crypto) via maximum likelihood on 10,800 samples (3 hours at 1-second intervals), refitted hourly. Currently in shadow mode — logging forecasts and computing MZ R^2 blend weights, but not affecting live trading decisions.

HAR-RV Model (Shadow Mode)

The Heterogeneous Autoregressive model of Realized Volatility captures multi-horizon persistence:

$$RV_{t+1} = \beta_0 + \beta_d RV_t^{(d)} + \beta_w RV_t^{(w)} + \beta_m RV_t^{(m)} + \varepsilon_t$$

Extended variants include jump components (HAR-J), semi-variance (HAR-Semi), implied volatility integration (HAR-IV), and variance risk premium (HAR-VRP). Fitted with ridge regression ($\lambda=0.05$) to prevent overfitting. Currently in shadow mode and **not producing viable models** — all variants are rejected by coefficient validity checks (negative weights, sum-of-weights outside $[0.3, 2.5]$). The 15-minute crypto environment may lack the daily/weekly seasonality HAR was designed for.

Adaptive Jump Detection

Jumps — sudden, large price moves — invalidate smooth volatility assumptions. The system uses an adaptive threshold rather than the fixed 3 σ approach:

- **EWMA variance tracking:** Tracks running variance of 15-second subsampled returns ($\lambda=0.94$)
- **Percentile-based threshold:** Jump trigger set at the asset’s own volatility distribution percentile, adapting to current regime
- **Tiered response:** Jump multiplier and cooldown duration scale with jump severity
- **Health monitoring:** Logs EWMA λ , percentile, threshold, and total jump count per asset

DVOL Integration

When Deribit implied volatility (DVOL) exceeds realized volatility by more than 50%, the system blends in the implied estimate using inverse-variance weighting. This respects the market’s forward-looking information during regime changes while anchoring to observed data.

Cross-Asset Beta

For assets without direct DVOL data (SOL, XRP), the system estimates a cross-asset beta against BTC using a 60-return lookback window, clamped to $[0.5, 3.0]$, allowing derivative signals to propagate across correlated assets.

3.2 Probability Model

Given the blended volatility, the probability engine estimates the likelihood that an asset stays above its threshold for the remaining window duration.

Step 1: Z-Score Computation

$$z = \frac{\text{threshold} - \text{spot}}{\text{spot} \times \sigma_{\text{blended}} \times \sqrt{T/5}}$$

where T is seconds remaining and σ_{blended} is per-5-second scale. The denominator represents the expected magnitude of price movement over the remaining window.

Step 2: NIG CDF (Per-Asset Fitted)

The model uses the Normal Inverse Gaussian distribution with per-asset fitted parameters:

$$p_{\text{raw}} = 1 - F_{\text{NIG}}(z; a, b, \mu, \delta)$$

where a controls tail heaviness, b captures asymmetry (skew), μ is location, and δ is scale. NIG parameters are fitted via maximum likelihood on 7 days of 60-second returns (~10,000 samples per asset) and stored in `dist_config.json`.

Why NIG over Student-t? NIG provides two key improvements: - **Asymmetry**: The b parameter captures the empirical skew in crypto returns (e.g., BTC $b = -0.019$, slight left skew) - **Better tail fit**: KS test p-values for NIG are dramatically higher (BTC: 0.11, ETH: 0.42) compared to Student-t (effectively 0), indicating NIG genuinely captures the return distribution

The system falls back to Student-t(df=4) if NIG parameters are unavailable.

Step 3: Data-Driven Calibration

Raw probabilities are calibrated using a CalibrationEngine that learns from settlement outcomes:

Method	Min Samples	Description
Fixed logistic (=0.85)	0	Default fallback — compresses extreme probabilities
Platt Scaling	200	2-parameter logistic (A, B) fitted to outcomes
Beta Calibration	350	3-parameter (a, b, c) — more flexible than Platt
Bayesian Linear Regression	50	Online posterior with prior w=1, b=0 (identity calibration)

The engine automatically promotes to better methods as data accumulates, with validation checks to prevent degradation.

Step 4: Dynamic Probability Cap

A time-dependent cap adjusts confidence based on time remaining:

Time to Expiry	Cap
> 10 minutes	93%
5–10 minutes	95%
2–5 minutes	97%
1–2 minutes	98.5%
< 1 minute	99.5%

As expiry approaches and less can go wrong, the cap relaxes to allow higher-confidence trades in the endgame.

Step 5: Market-Price Blending

For market prices below 96¢, the calibrated probability is blended with the market-implied probability:

$$p_{final} = 0.50 \times p_{cal} + 0.50 \times p_{market}$$

At 96¢ and above, blending is skipped to preserve edge in high-confidence endgame scenarios.

Sanity Checks

- **Z-score limit:** If $|z| > 12$, the market is refused (volatility estimate is likely wrong at extremes)
- **Model-market discrepancy:** If $p_{cal} > 90\%$ but market price $< 75\text{¢}$, the market is refused (the model may be missing information the market has)

3.3 Edge Detection

A trade requires positive expected value after accounting for fees.

Fee Formula

Kalshi charges fees using a variance-based formula:

$$\begin{aligned}\text{taker fee} &= \lceil 0.07 \times C \times P \times (1 - P) \rceil \text{ cents} \\ \text{maker fee} &= \lceil 0.0175 \times C \times P \times (1 - P) \rceil \text{ cents}\end{aligned}$$

where C is the number of contracts and P is the trade price as a decimal. The ceiling is applied to the total, not per contract.

Fee-Adjusted Edge

The scanner evaluates edge using taker fees (worst-case), so any candidate that passes the filter is profitable even if maker order is rejected:

$$\text{edge} = p_{\text{final}} - \frac{\text{best_ask}}{100} - \frac{\text{taker_fee}}{C \times 100}$$

A trade must satisfy:

$$\text{edge} \geq \text{MIN_EDGE_PCT} = 1.0\%$$

3.4 Order Flow Analysis

The OrderFlowEngine combines two signal sources to adjust the base probability:

Cross-Exchange Consensus

Real-time prices from Binance, Kraken, and Bybit are compared against Coinbase:

Signal	Condition	Adjustment
Strong consensus (favorable)	3+ exchanges lead by 0.3%	+2 pp
Single-exchange lead (favorable)	1 exchange leads by 0.2%	+1 pp
Strong consensus (opposing)	3+ exchanges oppose by 0.3%	-2 pp

Funding Rate Signals

Perpetual futures funding rates from CoinGlass indicate leverage buildup:

Signal	Condition	Adjustment
Extreme funding	Rate > 0.05% per 8h	-1.5 pp
Elevated funding	Rate > 0.03% per 8h	-0.5 pp

Kalshi Order Flow (Shadow Mode)

The KalshiOrderFlowTracker monitors Kalshi's own orderbook for predictive signals:

- **Imbalance:** Ratio of bid vs. ask depth — strong imbalance (>0.8 or <0.2) suggests directional pressure
- **Depth velocity:** Rate of change in total depth — draining liquidity may predict a move
- **Spread convergence:** Narrowing spread + trending depth suggests informed trading
- **Adjustments:** ± 1 to 1.5pp based on signal strength, with confidence levels based on snapshot count

Currently logging only — signals are computed but do not affect trading decisions. Needs ~200+ signal→settlement pairs to evaluate predictive power.

Total Adjustment Cap

All adjustments are summed and capped at **± 3 percentage points**, preventing any single signal source from dominating the probability estimate.

3.5 Execution Strategy

The executor uses a maker-first approach with three-tier `post_only` rejection handling and time-aware escalation.

Three-Tier Post-Only Handler

When a `post_only=True` maker order is rejected (the order would cross the spread rather than rest on the book), the system escalates through three tiers:

Tier	Trigger	Action
Tier 1: Normal maker	0–1 rejections	Standard maker order, 1–2¢ below fair value
Tier 2: Degraded maker	2 rejections	Same offset + 1¢ additional discount. If price drops below 86¢ floor, skipped.
Tier 3: Taker IOC	3+ rejections	Edge re-verified with actual taker fees → IOC order if still profitable

Rejection counts expire after 30 seconds and are per-ticker (unique per market window).

Time-Based Escalation

For orders that are successfully placed but sit unfilled:

Urgency	Time to Close	Maker Wait
Low	60–300s	15s
Medium	30–60s	10s
High	<30s	5s

Maker-to-Taker Conversion

1. Place maker order with `post_only=True` (guarantees maker fees, 4× cheaper)
2. Monitor for fills via Kalshi WebSocket (zero API cost) with REST polling fallback
3. Poll queue position every ~5s for queue-aware escalation timing
4. If timeout reached without fill:
 - Attempt `amend_order()` to convert to taker price in-place (avoids cancel+replace race)
 - If amend fails, fall back to cancel + IOC (`time_in_force="ioc"`) taker order
 - Re-validate price still in [86¢, 99¢] before taker submission

Partial Fill Handling

Orders may partially fill (e.g., 3 of 13 contracts). The execution engine tracks `filled_so_far` cumulatively and keeps the order active until fully filled or escalated. REST fill detection uses a `_seen_fill_ids` set to prevent double-counting across consecutive polls and against WebSocket fills.

UUID Persistence

Each order gets a `client_order_id` (UUID4) written to SQLite before API submission. This ensures crash recovery — if the bot restarts mid-order, it can reconcile using the persisted UUID.

3.6 Position Sizing

Quarter-Kelly Formula

The Kelly criterion maximizes long-run growth rate. The bot uses a quarter-Kelly fraction for safety:

$$f = 0.25 \times \frac{b \times p - q}{b}$$

where:

- $b = \frac{100 - \text{price}}{\text{price}}$ (net odds)
- p = win probability
- $q = 1 - p$

The dollar position is:

$$\text{contracts} = \left\lfloor f \times \frac{\text{bankroll}}{\text{price}} \right\rfloor$$

with a safety ceiling of `MAX_RISK_PER_TRADE` (50%) of bankroll.

Edge-Based Sizing Tiers

Thresholds are fee-adjusted (gross edge minus ~1¢ taker fee per contract):

Fee-Adjusted Edge	Risk Fraction	Approx Gross Edge
4%	50% of bankroll	~5%+
2%	35% of bankroll	~3%+
1.5%	20% of bankroll	~2.5%+
1%	10% of bankroll	~2%+

Drawdown Scaling

Balance vs. Starting	Sizing Adjustment
90%	Full quarter-Kelly
80–90%	Half of quarter-Kelly
< 80%	Quarter of quarter-Kelly

This creates a geometric de-risking curve that preserves capital during losing streaks.

Part 4: Risk Management

Position Sizing Controls

- **Quarter-Kelly:** Conservative fraction ($0.25\times$) of the theoretically optimal bet size
- **Drawdown scaling:** Size halved below 90% of starting balance, quartered below 80%
- **Hard limits:** Maximum risk per trade capped at 50% of bankroll

Market Selection Controls

- **Multi-asset capable:** Can trade multiple assets per 15-minute window
- **Price range guardrails:** Only trade contracts priced 86–99¢ — below 86¢ has historically poor win rates; above 99¢ offers insufficient reward
- **Minimum edge threshold:** Fee-adjusted edge must exceed 1.0% after taker fees (worst-case)
- **Scanner uses taker fees:** Every candidate is profitable even if forced to taker execution

Model Sanity Controls

- **Z-score limit:** Refuse markets where $|z| > 12$ (extreme inputs suggest volatility estimate is wrong)
- **Model-market discrepancy:** If the model estimates $>90\%$ probability but the market prices below 75¢, refuse (the model may be missing material information)
- **Dynamic probability cap:** Time-dependent ceiling (93–99.5%) prevents overconfidence regardless of model output
- **Data-driven calibration:** CalibrationEngine learns from settlement outcomes, replacing fixed assumptions with empirical mappings

Execution Controls

- **Three-tier post_only handler:** Escalates from normal maker $\rightarrow$ degraded maker $\rightarrow$ taker IOC after repeated rejections, with edge re-verification at each tier
- **Maker-first with post_only:** Guarantees maker fee tier (75% cheaper), rejected if it would cross the spread
- **WebSocket fill detection:** Zero-cost fill monitoring via Kalshi WebSocket, with REST polling fallback
- **Amend-first escalation:** Uses `amend_order()` API to convert maker $\rightarrow$ taker in-place, avoiding cancel+replace race conditions

- **IOC taker orders:** Taker escalation uses `time_in_force="ioc"` (immediate-or-cancel) to prevent stale resting orders
- **Price re-validation:** After maker timeout, the system re-fetches the orderbook and re-validates the price range before submitting a taker order
- **UUID persistence:** Order IDs written to disk before API submission, enabling crash recovery without duplicate orders
- **Rejection expiry:** `Post_only` rejection counts expire after 30 seconds, preventing stale state from affecting future windows

Part 5: Performance & Observations

This section contains live statistics from the bot's observation database, updated on each whitepaper build.

Overview

Metric	Value
Total markets evaluated	0
Total markets settled	0
Observation period	N/A
Assets tracked	BTC, ETH, SOL, XRP

Filter Pipeline Breakdown

Of all evaluated markets, here is how they were classified:

Filter Stage	Count	Percentage
Low probability	0	0%
No orderbook	0	0%
No best ask	0	0%
Price out of range	0	0%
Insufficient edge	0	0%
Zero sizing	0	0%
Strategy wait	0	0%
Candidate (passed all filters)	0	0%

Hypothetical Performance

Based on observation-mode tracking of markets that passed all filters:

Metric	Value
Total hypothetical trades	0
Observation P&L (cents)	0

Metric	Value
Win rate	N/A

Win Rate by Price Bucket

Entry Price	Trades	Wins	Win Rate
80–84¢	0	0	—
85–89¢	0	0	—
90–94¢	0	0	—
95–99¢	0	0	—

Part 6: Infrastructure

Deployment

- **Host:** DigitalOcean droplet (Ubuntu 24.04)
- **Runtime:** Python 3, virtualenv
- **Process manager:** systemd (`kalshi-bot` service)
- **Startup:** `start.sh` activates `venv`, sources `.env`, launches `bot.py`
- **Auto-deploy:** Pushing to `main` triggers a GitHub Action that SSHes into the VPS, pulls the latest code, syntax-checks `bot.py`, and restarts the service

Data Persistence

SQLite (`state.db`)

The primary state store uses SQLite in WAL (Write-Ahead Logging) mode for crash resilience:

Table	Purpose
<code>positions</code>	Active positions (ticker, asset, side, count, avg price)
<code>pending_orders</code>	Orders awaiting fill (with UUID <code>client_order_id</code>)
<code>settled_trades</code>	Completed trades with P&L
<code>rejected_opportunities</code>	Markets rejected with reason and model state
<code>evaluated_opportunities</code>	Every market evaluation with filter stage (UNIQUE on ticker + stage)

JSONL Journals

Nine append-only journal files provide a complete audit trail:

1. **`scan_journal`** — Every tick: price snapshots, orderbook depth
2. **`opportunity_journal`** — Every market evaluation with filter stage

3. **rejection_journal** — Settlement outcomes for rejected opportunities
4. **trade_journal** — Order fills with entry/exit details
5. **settlement_journal** — Outcome determination and P&L
6. **order_journal** — Full order lifecycle (create, cancel, fill)
7. **execution_journal** — Maker-to-taker escalation events
8. **performance_journal** — Session summaries and strategy counts
9. **fill_model_journal** — Maker order lifecycle data (queue position, time-to-fill, spread at submission) for future ML fill prediction model

Firestore Real-Time Dashboard

A Firestore integration provides a live web dashboard showing:

- Current positions and P&L
- Active market evaluations
- Volatility regime indicators
- Order flow signals
- Execution engine statistics (amend success rate, WS fill ratio, post_only rejection counts, taker escalation counts)

Shadow Mode Features

The system supports shadow mode for experimental features — they compute and log but do not affect live trading decisions:

Feature	Status	Readiness
EGARCH blend	Shadow	Closest to promotion (R^2 0.42–0.61 typical)
EGARCH core vol	Shadow	Stable convergence, building block for blend
HAR-RV model	Shadow	Not viable — all model variants rejected
Kalshi order flow	Shadow	Early data collection, needs 200+ outcomes

Promoted features (shadow off, driving live behavior): - **Adaptive jump detection** — percentile-based thresholds per asset - **Adaptive RK bandwidth** — data-driven H^* selection

Generated: N/A